

AgentOps OS

Système d'exploitation pour agents de codage auto-améliorants sous supervision humaine - Usine d'intelligence logicielle.

Nature	Portée	Date
Spécification théorique et technique totale	Projet, IDE, app, entreprise, système autonome	27 avril 2026

Contrôle du document

Champ	Valeur
Titre	AgentOps OS - Spécification théorique et technique totale
Version	1.0
Statut	Document fondateur complet
Objectif	Définir une méthode et une architecture totale pour agents de codage auto-améliorants sous supervision humaine.
Non-objectif	Ne pas décrire un simple MVP ; ne pas autoriser l'auto-modification non supervisée.
Principe de sécurité	Toute autonomie doit être bornée par permissions, sandbox, audit, evidence, évaluation et validation humaine.

Lecture recommandée

Lire les sections 1 à 11 pour la méthode, les sections 12 à 32 pour l'architecture technique et les annexes pour les modèles directement réutilisables.

Table des matières

0. Résumé exécutif	5
1. Thèse fondatrice	7
2. Définitions normatives	8
3. Axiomes de conception	9
4. Modèle de l'usine d'intelligence	10
5. Positionnement stratégique	11
6. Méthode AOS	13
7. Cycle standard d'une mission	14
8. Format canonique d'une mission	15
9. Taxonomie des agents	16
10. Niveaux d'autonomie	17
11. Règles d'incorporation	18
12. Architecture globale	20
13. Composants fonctionnels	21
14. Standard .agentops	22
15. agentops.yml	23
16. Modèle de données	24
17. États d'une mission	25
18. Journal d'audit	26
19. Policy Engine	27
20. Sandbox et exécution	28
21. Gestion des permissions	29
22. Protocole d'auto-amélioration supervisée	30
23. ImprovementProposal	31
24. Memory Engine	32
25. Evaluation Engine	33
26. Human Approval Layer	34
27. Security Model	35
28. Vendor-neutral Agent Layer	36
29. API technique cible	37
30. CLI cible	38
31. Intégration IDE	39
32. Déploiements et topologies	40
33. Observabilité	41
34. Workflows opérationnels	43
35. Workflow Feature complet	44
36. Workflow Bugfix complet	45
37. Workflow Self-Improve complet	46

38. Maturité organisationnelle	47
39. Gouvernance et rôles humains	48
40. Anti-patterns	49
41. Doctrine qualité 10/10	50
A. Annexe A - Mission Report	52
B. Annexe B - Schéma JSON Mission	53
C. Annexe C - Checklists	54
D. Annexe D - Références	55

0. Résumé exécutif

La spécification en une page

AgentOps OS est une couche de gouvernance, d'orchestration et d'amélioration continue pour agents de codage. Elle transforme un projet logiciel en environnement agentique contrôlé : missions structurées, agents spécialisés, permissions, sandbox, tests, preuves, mémoire et auto-amélioration supervisée.

Thèse centrale

La superintelligence opérationnelle ne commence pas par un modèle qui sait tout. Elle commence par un système capable de transformer une intention en artefact exécutable, de tester cet artefact, de capitaliser l'apprentissage, puis d'améliorer la boucle elle-même.

Ce que le système produit	Ce que le système contrôle
Code, scripts, tests, documentation, rapports, agents spécialisés, workflows.	Permissions, risques, validations, audit, sandbox, réseau, secrets, mémoire.
Améliorations proposées du système agentique.	Interdiction d'auto-élévation de privilèges et validation humaine obligatoire.
Une usine d'intelligence cumulative.	Une chaîne de preuves et de responsabilité.

La spécification vise le système cible complet, pas un MVP. Elle est conçue pour être incorporable dans chaque projet via un noyau .agentops, utilisable dans un IDE, pilotable par CLI/API et exploitable comme application indépendante.

PARTIE I

Théorie de l'usine d'intelligence

1. Thèse fondatrice

Pourquoi les agents de codage deviennent l'infrastructure opératoire de l'intelligence artificielle avancée

La capacité de codage n'est pas une compétence parmi d'autres : c'est la capacité de transformer une intention en système exécutable. Un agent qui maîtrise le code peut lire un environnement, modifier des fichiers, configurer des outils, lancer des tests, corriger des erreurs, documenter ses décisions et proposer de nouvelles procédures.

Le codage est donc le premier langage opératoire de l'intelligence numérique. Le langage naturel décrit ; le code exécute. Un agent de codage relie ces deux mondes : il convertit une intention humaine en action vérifiable dans un système réel.

Principe directeur

AgentOps OS ne cherche pas seulement à faire coder plus vite. Il cherche à rendre l'action agentique fiable, contrôlable, mesurable, mémorisable et améliorable.

Idée	Conséquence
Comprendre un système informatique implique de pouvoir le modifier.	L'agent de codage devient un outil de compréhension active.
Modifier sans tester n'est pas agir intelligemment.	L'évaluation devient centrale.
Apprendre sans gouvernance peut amplifier les erreurs.	La mémoire doit être contrôlée.
S'améliorer sans supervision peut devenir une dérive.	L'auto-amélioration doit être validée et réversible.

2. Définitions normatives

Vocabulaire utilisé par toute la spécification

Terme	Définition opératoire
Agent de codage	Système IA capable de lire, modifier ou générer des artefacts logiciels et d'utiliser des outils de développement.
Agent auto-améliorant	Agent ou système agentique capable de proposer des modifications de ses instructions, workflows, outils, tests, mémoire ou architecture.
Supervision humaine	Ensemble des points où un humain valide, refuse, corrige ou limite une action agentique.
Usine d'intelligence	Chaîne de production où des intentions deviennent artefacts exécutables, testés, documentés et capitalisés.
Mission	Unité de travail structurée, évaluée et traçable.
Evidence	Preuve attachée à une action : diff, log, test, capture, rapport, hash, approbation.
Noyau .agentops	Répertoire portable contenant règles, agents, permissions, workflows, mémoire, évaluations et logs.
Gouvernance	Système de limites, rôles, règles, audits, validations et mécanismes de rollback.

Ces définitions sont normatives : elles rendent AgentOps OS indépendant d'un modèle, d'un fournisseur ou d'un IDE particulier.

3. Axiomes de conception

Les lois minimales du système

Axiome	Formulation	Conséquence
A1 - Action vérifiable	Toute action agentique doit produire une preuve.	Pas de modification sans diff, log ou résultat vérifiable.
A2 - Autorité distribuée	Aucun agent ne concentre lecture, écriture, exécution, approbation et auto-modification.	Séparation stricte des rôles.
A3 - Mémoire contrôlée	La mémoire augmente la performance mais peut amplifier les erreurs.	Toute mémoire doit être classée, datée et révisable.
A4 - Auto-amélioration bornée	Un agent peut proposer sa propre amélioration, mais ne s'accorde pas plus de pouvoir.	Contrôle humain sur règles et permissions.
A5 - Reproductibilité	Une action importante doit être rejouable ou explicable.	Logs, versions, commandes, environnement, hashes.
A6 - Dégradabilité	Le système reste utile si un fournisseur devient indisponible.	Architecture vendor-neutral.

Formule courte

AgentOps OS n’est pas un agent. C’est le système qui rend des agents utiles, sûrs, mesurables et améliorables.

4. Modèle de l'usine d'intelligence

Transformer le travail cognitif en chaîne de production agentique

L'usine d'intelligence est une chaîne de valeur dont l'entrée est une intention humaine : bug à corriger, fonctionnalité à créer, rapport à produire, migration à effectuer, architecture à auditer. La sortie n'est pas seulement du code ; c'est un ensemble d'artefacts validés.

Intention humaine

-> Mission structurée

-> Plan agentique

-> Action contrôlée

-> Tests et preuves

-> Validation humaine

-> Intégration

-> Mémoire projet

-> Proposition d'amélioration

Entrée	Transformation	Sortie attendue
Besoin flou	Clarification et cadrage	Mission complète
Code existant	Cartographie et compréhension	Modèle du système
Erreur	Reproduction et diagnostic	Correctif minimal et test anti-régression
Répétition	Automatisation	Workflow ou script réutilisable
Échec agentique	Analyse post-mortem	Nouvelle règle, test ou garde-fou

5. Positionnement stratégique

Ce que le système est et n'est pas

AgentOps OS est	AgentOps OS n'est pas
Un système d'exploitation logique pour agents de codage.	Un simple chatbot de développement.
Une couche de gouvernance et d'orchestration.	Un modèle de langage propriétaire.
Un standard portable par projet.	Un IDE unique fermé.
Un moteur d'audit, de mémoire et d'évaluation.	Un générateur de code sans contrôle.
Une doctrine d'auto-amélioration supervisée.	Une autorisation d'auto-modification illimitée.

L'opportunité n'est pas de remplacer tous les agents existants. L'opportunité est de créer la couche supérieure qui les rend composables, gouvernables, comparables et industrialisables.

PARTIE II

Méthode AOS incorporable dans chaque projet

6. Méthode AOS

Méthode universelle incorporable dans chaque projet

La méthode AOS impose qu'un projet compatible agentique possède un noyau de gouvernance local. Ce noyau est lisible par un agent, par un IDE, par un orchestrateur web ou par un humain. Il explicite ce que le projet veut, ce qu'il interdit, ce qu'il sait, ce qu'il teste et ce qu'il apprend.

```
.agentops/  
agentops.yml  
mission.schema.json  
rules.md  
permissions.yml  
memory/  
agents/  
workflows/  
evals/  
audit/  
reports/  
approvals/  
policies/  
adapters/
```

Le dossier .agentops est le système nerveux local du projet. Il ne contient pas forcément le produit, mais il contient la manière dont les agents doivent le comprendre, l'améliorer et rendre compte de leurs actions.

7. Cycle standard d’une mission

Le protocole qui empêche l’improvisation agentique

Étape	Acteur principal	Sortie obligatoire
1. Intention	Humain	Besoin brut
2. Structuration	Mission Planner	Mission formalisée
3. Cartographie	Architecte	Zone d’impact
4. Plan	Architecte + Reviewer	Plan de changement
5. Autorisation	Humain ou policy engine	Décision d’exécution
6. Exécution	Coder / Worker	Diff ou artefact
7. Vérification	Tester / Security	Résultats de tests et risques
8. Revue	Reviewer + Humain	Acceptation, rejet ou correction
9. Intégration	SCM Adapter	Commit, PR ou release
10. Apprentissage	MetaDev + Memory	Mémoire, règle ou workflow mis à jour

Règle de base

Une mission qui ne peut pas être vérifiée ne doit pas être automatisée à un niveau d’autonomie élevé.

8. Format canonique d'une mission

Contrat entre humain, agent et système

```
mission_id: AOS-MIS-2026-0001
title: Ajouter un mécanisme de validation de formulaire
intent: "Réduire les erreurs utilisateur avant soumission"
context:
product_area: onboarding
repository: app-web
branch: feature/form-validation
scope:
include:
- app/onboarding/**
- components/forms/**
exclude:
- auth/**
- database/migrations/**
constraints:
max_files_changed: 12
no_new_dependencies: true
no_database_change: true
success_criteria:
- "Build sans erreur"
- "Tests unitaires existants passent"
- "Validation email + champs obligatoires"
risk_level: medium
autonomy_level: 4
human_approval:
before_execution: true
before_merge: true
before_policy_change: true
```

La mission est une unité de travail auditable. Elle contient assez d'information pour être planifiée, exécutée, testée et expliquée, sans donner carte blanche à l'agent.

9. Taxonomie des agents

Rôles spécialisés de l'usine d'intelligence

Agent	Mission	Permissions par défaut	Anti-dérive
Planner	Transformer intention en mission.	Lecture + écriture mission.	Ne modifie pas le code.
Architect	Comprendre architecture et impacts.	Lecture globale.	Pas d'exécution.
Coder	Implémenter changements autorisés.	Lecture + écriture limitée.	Pas de secrets, pas de déploiement.
Tester	Vérifier build, tests, régressions.	Lecture + exécution contrôlée.	Pas d'écriture produit hors rapports.
Security	Analyser vulnérabilités et permissions.	Lecture + scanners.	Ne corrige pas sans mission.
Reviewer	Juger qualité et maintenabilité.	Lecture + commentaires.	Ne merge pas.
Documenter	Mettre à jour docs et changelog.	Écriture docs.	Ne touche pas logique métier.
MetaDev	Proposer amélioration agentique.	Lecture logs + propositions.	Validation humaine obligatoire.
Release	Préparer livraison.	CI/CD limité.	Déploiement approuvé.

Chaque agent est un rôle, pas forcément un modèle distinct. Un même modèle peut jouer plusieurs rôles, mais AgentOps OS conserve les séparations logiques.

10. Niveaux d'autonomie

Escalier de délégation contrôlée

Niveau	Nom	Capacité autorisée	Validation
0	Observation	Lire et expliquer.	Aucune écriture.
1	Proposition	Proposer plan, risques, fichiers.	Validation avant suite.
2	Brouillon	Générer patch non appliqué.	Validation avant application.
3	Écriture locale	Modifier une branche de travail.	Validation avant tests étendus.
4	Écriture + tests	Modifier et lancer tests autorisés.	Validation avant PR.
5	PR supervisée	Ouvrir PR avec rapport complet.	Review humaine.
6	Merge conditionnel	Merge si politiques et reviews satisfaites.	Approbation stricte.
7	Release supervisée	Préparer release et checklist.	Release manager.
8	Auto-amélioration proposée	Proposer règles, prompts, workflows.	Validation humaine.
9	Auto-amélioration contrôlée	Appliquer changements agentiques approuvés.	Double validation + rollback.

Limite non négociable

Un agent ne peut pas élever son propre niveau d'autonomie. Seule une gouvernance humaine ou une politique signée peut le faire.

11. Règles d'incorporation

Comment rendre un projet compatible AgentOps OS

1. Créer le répertoire `.agentops` à la racine du dépôt.
2. Définir `agentops.yml` comme constitution locale du projet.
3. Ajouter `rules.md` pour les conventions humaines et techniques.
4. Déclarer les permissions par agent et par type d'action.
5. Définir au moins trois workflows : `feature`, `bugfix`, `review`.
6. Connecter les tests obligatoires à `evals/quality.yml`.
7. Initialiser `memory/project.md` avec `stack`, décisions et erreurs récurrentes.
8. Activer `audit/action-log.jsonl` pour toute opération agentique.
9. Exiger approbation humaine pour secrets, auth, base de données, production et auto-amélioration.
10. Évaluer périodiquement les performances de l'usine d'intelligence.

La méthode doit rester légère pour un petit projet mais suffisamment stricte pour une entreprise. Le même noyau doit pouvoir évoluer vers plus de gouvernance sans changer de philosophie.

PARTIE III

Spécification technique totale

12. Architecture globale

Vue système complète d'AgentOps OS

```
Interface Layer
Web App | IDE Extension | CLI | API | Chat Gateway
↓
Mission Layer
Mission Manager | Human Approval Console | Work Queue
↓
Orchestration Layer
Agent Router | Policy Engine | Capability Registry | Tool Registry
↓
Execution Layer
Sandbox Executor | Repo Adapter | CI Adapter | Runtime Nodes
↓
Verification Layer
Tests | Security | Evaluation | Regression | Evidence Store
↓
Knowledge Layer
Memory Engine | Lessons | Decisions | Knowledge Graph
↓
Governance Layer
Audit | RBAC | Compliance | Rollback | Risk Register
```

L'architecture sépare interfaces, orchestration, exécution, évaluation, mémoire et gouvernance. Le modèle de langage ne doit jamais devenir le point central de confiance. Le modèle propose et agit dans un périmètre ; le système encadre, vérifie et conserve la preuve.

13. Composants fonctionnels

Modules obligatoires du système cible

Module	Responsabilité	Interfaces
Mission Manager	Créer, suivre, prioriser et fermer les missions.	Web, API, CLI
Agent Router	Choisir agent, modèle et outils selon mission et risque.	Policy Engine, Capability Registry
Policy Engine	Décider ce qui est autorisé, bloqué ou soumis à validation.	Permissions, RBAC, audit
Sandbox Executor	Exécuter commandes en environnement contrôlé.	Container, OS sandbox, proxy réseau
Repo Adapter	Lire branches, commits, PR, diffs, tags.	Git, GitHub, GitLab, local FS
Tool Registry	Déclarer outils disponibles, risques, entrées, sorties.	MCP, CLI, API internes
Evaluation Engine	Calculer scores qualité, risque, confiance, coût.	Tests, métriques, rapports
Evidence Store	Conserver diffs, logs, résultats, approbations.	Objet, DB, hash ledger
Memory Engine	Capitaliser apprentissages et décisions.	Markdown, vector store, graph DB
MetaDev Engine	Détecter améliorations du système agentique.	Audit logs, evals, mémoire
Governance Console	Superviser, approuver, bloquer, réviser.	RBAC, audit, dashboard

Ces modules forment la cible totale. Ils peuvent être déployés progressivement, mais la spécification les définit dès le départ pour éviter une architecture prisonnière d'un MVP.

14. Standard .agentops

Structure du noyau portable

```
.agentops/  
agentops.yml # Constitution du projet agentique  
README.md # Mode d'emploi humain  
rules.md # Règles générales du projet  
policies/  
autonomy.yml # Niveaux autorisés  
approvals.yml # Matrice d'approbation  
security.yml # Secrets, réseau, fichiers interdits  
data.yml # Données sensibles et conformité  
agents/  
architect.agent.yml  
coder.agent.yml  
tester.agent.yml  
security.agent.yml  
reviewer.agent.yml  
metadev.agent.yml  
workflows/  
feature.workflow.yml  
bugfix.workflow.yml  
refactor.workflow.yml  
migration.workflow.yml  
self-improve.workflow.yml  
evals/  
quality.yml  
security.yml  
performance.yml  
regression.yml  
human-acceptance.yml  
memory/  
project.md  
decisions.md  
lessons.md  
patterns.md  
incidents.md  
audit/  
action-log.jsonl  
approval-log.jsonl  
risk-log.jsonl  
reports/  
mission-reports/  
improvement-proposals/  
release-reports/
```

Ce standard doit cohabiter avec AGENTS.md, CLAUDE.md, .codex/, .github/ et les fichiers spécifiques aux IDE. AgentOps OS ne remplace pas ces conventions : il les orchestre.

15. agentops.yml

Constitution technique du projet

```
spec_version: "1.0"
project:
  id: "com.example.agentops.demo"
  name: "Demo App"
  type: "web_application"
  criticality: "medium"
  owners:
    product: "product-owner@example.com"
    technical: "tech-lead@example.com"
    security: "security@example.com"

runtime:
  default_branch: "main"
  protected_branches: ["main", "production"]
  package_manager: "npm"
  build_commands:
    - "npm run typecheck"
    - "npm run test"
    - "npm run build"

agent_policy:
  default_autonomy_level: 2
  max_autonomy_level: 5
  require_human_approval_for:
    - "secrets_access"
    - "database_schema_change"
    - "auth_change"
    - "production_deploy"
    - "policy_change"
    - "self_improvement_apply"

memory:
  enabled: true
  review_cycle_days: 30
  forbidden_memory:
    - "raw_secrets"
    - "personal_sensitive_data"
```

agentops.yml doit être court, strict et lisible. Les détails volumineux appartiennent aux sous-dossiers politiques, workflows, evals et agents.

16. Modèle de données

Entités fondamentales

Entité	Champs clés	Rôle
Project	id, name, criticality, owners, repos, policies	Contexte global
Mission	id, title, status, scope, risk, autonomy, evidence	Unité de travail
Agent	id, role, capabilities, model, tools, permissions	Acteur logique
Tool	id, command/api, input_schema, output_schema, risk	Capacité externe
Policy	rules, conditions, enforcement, exceptions	Contrôle d'action
Action	agent, command, files, diff, result, timestamp	Trace atomique
Approval	who, what, decision, reason, signature	Validation humaine
MemoryItem	type, content, source, confidence, expiry	Apprentissage
Evaluation	tests, scores, findings, recommendation	Jugement mesuré
ImprovementProposal	target, change, rationale, expected_gain, risk	Auto-amélioration proposée

Le système traite ces entités comme des objets persistants et versionnés. Une mission terminée sans preuves attachées doit être considérée incomplète.

17. États d'une mission

Machine d'état obligatoire

```
DRAFT
-> PLANNED
-> WAITING_APPROVAL
-> APPROVED
-> IN_PROGRESS
-> TESTING
-> SECURITY_REVIEW
-> HUMAN_REVIEW
-> READY_TO_MERGE
-> MERGED
-> LEARNING_CAPTURED
-> CLOSED

Transitions alternatives:
ANY -> BLOCKED
ANY -> CANCELLED
IN_PROGRESS -> ROLLED_BACK
HUMAN_REVIEW -> REVISION_REQUIRED
TESTING -> FAILED_TESTS
SECURITY_REVIEW -> RISK_REJECTED
```

Chaque transition écrit un événement dans le journal. Les états déterminent permissions, notifications, SLA, obligations de preuve et règles de clôture.

18. Journal d'audit

Traçabilité inviolable et compréhensible

```
{
  "event_id": "evt_01HZZZ",
  "timestamp": "2026-04-27T10:15:31Z",
  "project_id": "com.example.agentops.demo",
  "mission_id": "AOS-MIS-2026-0001",
  "agent_id": "coder.worker.01",
  "event_type": "file_modified",
  "autonomy_level": 4,
  "scope": {"files": ["components/forms/EmailField.tsx"]},
  "reason": "Add email validation required by mission success criteria",
  "diff_hash": "sha256:...",
  "policy_decision": "allowed",
  "human_approval_id": "appr_...",
  "result": "success"
}
```

- Chaque événement est append-only.
- Les logs sensibles sont masqués sans détruire la preuve.
- Les diffs importants sont hashés.
- Les approbations sont liées à la mission et à la politique applicable.
- Les logs permettent un post-mortem humain sans reconstruire toute la conversation agentique.

But de l'audit

L'audit ne sert pas seulement à surveiller. Il sert à apprendre, corriger, comparer les agents et construire la mémoire de l'usine.

19. Policy Engine

Système de règles avant action

Le Policy Engine ne demande pas au modèle s'il peut agir ; il évalue une action contre des règles déclaratives et retourne une décision : allow, deny, require_approval, require_sandbox, require_review ou require_more_context.

```
rule: deny_secrets_read
when:
  action.type: "file_read"
  path.matches:
    - ".env"
    - ".env.*"
    - "secrets/**"
then:
  decision: "deny"
  reason: "Agents cannot read raw secrets"

rule: require_approval_for_auth_changes
when:
  action.type: "file_write"
  path.matches:
    - "auth/**"
    - "middleware/auth*"
then:
  decision: "require_approval"
  approver_role: "security_owner"
```

Décision	Effet
allow	Action autorisée et journalisée.
deny	Action bloquée avec raison.
require_approval	Pause jusqu'à validation humaine.
require_sandbox	Action possible uniquement en environnement isolé.
require_review	Action autorisée mais revue spécifique requise.
require_more_context	Action bloquée jusqu'à clarification.

20. Sandbox et exécution

Confinement technique des agents

Le sandbox est la frontière qui permet à un agent d’agir sans obtenir un accès illimité à la machine, au réseau, aux secrets ou à la production. AgentOps OS généralise ce principe dans une couche indépendante.

Dimension	Contrôle recommandé
Système de fichiers	Racine de travail limitée, chemins interdits, montage read-only pour zones sensibles.
Réseau	Off par défaut, allowlist de domaines, méthodes HTTP limitées.
Secrets	Jamais exposés en clair ; injection temporaire par broker si nécessaire.
Commandes	Allowlist par workflow, denylist globale, paramètres validés.
Durée	Timeouts stricts, quotas CPU/mémoire.
Dépendances	Installation contrôlée, lockfiles, scan licences/vulnérabilités.
Sorties	Filtrage PII/secrets, hash, archivage evidence.

Doctrine

Le modèle ne doit jamais être le seul mécanisme de sécurité. Les limites doivent être imposées par l’environnement, les politiques et les systèmes d’approbation.

21. Gestion des permissions

Contrôle par agent, mission, ressource et contexte

```
permissions:
  coder:
    read:
      allow: ["src/**", "components/**", "package.json"]
      deny: [".env*", "secrets/**", "infra/prod/**"]
    write:
      allow: ["src/**", "components/**", "tests/**"]
      require_approval: ["auth/**", "database/**"]
      deny: ["infra/prod/**"]
    execute:
      allow: ["npm run test", "npm run typecheck", "npm run build"]
      deny: ["rm -rf *", "curl * | sh", "ssh *"]
  metadev:
    write:
      allow: [".agentops/reports/improvement-proposals/**"]
      require_approval: [".agentops/policies/**", ".agentops/agents/**"]
```

Une permission est contextualisée : le même agent peut lire un fichier dans une mission d'audit mais pas dans une mission de génération de fonctionnalité. Le niveau d'autonomie, le risque, la branche et la criticité du projet influencent la décision.

22. Protocole d'auto-amélioration supervisée

Améliorer le système sans perdre la maîtrise

L'auto-amélioration ne signifie pas qu'un agent se réécrit librement. Elle signifie qu'un système observe ses propres performances, formule des hypothèses d'amélioration, les teste isolément et soumet les changements à la gouvernance.

1. Collecter les traces : missions, temps, erreurs, tests échoués, corrections humaines.
2. Détecter un motif : erreur récurrente, instruction ambiguë, workflow trop lent, test manquant.
3. Formuler une proposition : nouvelle règle, prompt, sous-agent, test, outil, seuil ou policy.
4. Classer le risque : faible, moyen, élevé, critique.
5. Créer une branche agentique ou un espace de simulation.
6. Évaluer avant/après sur un jeu de missions ou d'incidents représentatif.
7. Produire un rapport : gain attendu, risques, preuves, rollback.
8. Demander validation humaine selon la matrice d'autorité.
9. Appliquer le changement approuvé.
10. Surveiller les effets et revenir en arrière si les métriques se dégradent.

Verrou absolu

Les politiques de sécurité, la matrice d'approbation, les niveaux d'autonomie et les accès secrets ne peuvent être modifiés que par validation humaine explicite.

23. ImprovementProposal

Objet standard de l'auto-amélioration

```
improvement_proposal:  
  id: IMP-2026-0042  
  target: ".agentops/workflows/bugfix.workflow.yml"  
  change_type: "workflow_update"  
  rationale: "37% of bugfix missions failed because no reproduction step was  
  enforced"  
  proposed_change: "Make reproduction evidence mandatory before code modification"  
  expected_gain:  
    regression_risk_reduction: "medium"  
    review_time_reduction: "low"  
  risk: "low"  
  evaluation_plan:  
    replay_last_missions: 20  
    compare_failure_rate: true  
  rollback_plan:  
    revert_commit: true  
    restore_previous_workflow: true  
  approval_required_from:  
    - "technical_owner"
```

Une proposition d'amélioration doit contenir une cible précise, un gain attendu, un risque, un plan d'évaluation et un plan de rollback. Une proposition sans évaluation ne doit pas modifier le système.

24. Memory Engine

Mémoire utile, gouvernée et révisable

La mémoire transforme l’expérience en amélioration sans devenir un réservoir d’erreurs. Elle distingue faits stables, décisions humaines, patterns techniques, incidents, préférences de style et hypothèses temporaires.

Type de mémoire	Exemple	Durée	Révision
Stack	Next.js, TypeScript, Supabase	Longue	À chaque changement majeur
Décision	Pas de nouvelle dépendance sans accord	Longue	Mensuelle ou par owner
Pattern	Utiliser le composant Button existant	Moyenne	À chaque refactor UI
Incident	Build cassé par import circulaire	Longue	Post-mortem
Préférence	Ton de documentation direct et court	Moyenne	Révisable par humain
Hypothèse	Le bug vient probablement du cache	Courte	Expire automatiquement

25. Evaluation Engine

Mesurer la qualité de l'intelligence produite

AgentOps OS note chaque mission et chaque agent. Sans mesure, l'autonomie augmente par impression subjective. Avec mesure, elle augmente par preuve.

Score	Formule indicative	Signification
QualityScore	tests_passed × review_quality × maintainability	Qualité technique de la sortie.
RiskScore	impact × likelihood × exposure	Risque résiduel.
ConfidenceScore	evidence_strength × test_coverage × reviewer_agreement	Confiance dans le résultat.
CostScore	tokens + compute + human_review_time	Coût total réel.
AutonomyReadiness	quality_history - incident_penalty + evidence_maturity	Capacité à monter d'un niveau.
LearningYield	reusable_lessons / total_missions	Capitalisation produite.

```
mission_evaluation:
mission_id: AOS-MIS-2026-0001
scores:
quality: 0.86
security: 0.92
confidence: 0.81
risk: 0.18
cost_efficiency: 0.74
learning_yield: 0.42
decision: "approve_with_notes"
required_followups:
- "Add edge case for empty email"
- "Update memory with form validation pattern"
```

26. Human Approval Layer

Supervision humaine comme organe de contrôle

La supervision humaine est le mécanisme qui permet d’augmenter l’autonomie sans perdre la confiance. AgentOps OS sépare validation produit, validation technique, validation sécurité et validation gouvernance.

Action	Approbateur minimal	Double validation ?
Modification UI faible risque	Technical owner ou reviewer	Non
Nouvelle dépendance	Technical owner	Selon criticité
Changement auth	Security owner + technical owner	Oui
Migration base de données	Technical owner + data owner	Oui
Déploiement production	Release owner	Oui si critique
Changement de permissions agent	Governance owner	Oui
Auto-amélioration des politiques	Governance owner + security owner	Oui

```
approval:
id: appr_2026_0009
mission_id: AOS-MIS-2026-0001
approver: "tech-lead@example.com"
role: "technical_owner"
decision: "approved"
scope:
- "Apply patch to feature branch"
reason: "Plan is limited, tests are defined, no auth/database changes"
expires_at: "2026-04-28T18:00:00Z"
```

27. Security Model

Menaces, contrôles et résilience

Menace	Description	Contrôle AgentOps OS
Prompt injection	Instruction malveillante dans issue, doc, web ou dépendance.	Isolation sources non fiables, provenance, policy engine.
Exfiltration	Sortie de code, secrets ou données sensibles.	Réseau off par défaut, allowlist, DLP, secret broker.
Dépendance malveillante	Package ou script install contenant payload.	Lockfile, registry policy, scan, approbation.
Commande destructive	Suppression ou modification massive.	Denylist, approval, snapshots, rollback.
Mémoire corrompue	Apprentissage faux ou dangereux persistant.	Source, confidence, expiry, review humaine.
Escalade d'autonomie	Agent qui modifie ses propres permissions.	Interdiction par policy, double approbation.
Hallucination de preuve	Agent affirme avoir testé sans preuve.	Evidence store obligatoire, logs machines.
Supply chain	CI/CD, scripts, tokens, artefacts compromis.	Provenance, signatures, SBOM.

Le modèle de sécurité considère les agents comme capables d'erreur, de confusion contextuelle et d'obéissance excessive à des instructions non fiables. La confiance est donnée à la preuve et aux politiques, pas à la bonne intention du modèle.

28. Vendor-neutral Agent Layer

Compatibilité Claude Code, Codex, agents locaux et futurs systèmes

AgentOps OS fonctionne comme couche d’orchestration au-dessus des agents existants et futurs. Claude Code, Codex, Cursor, Copilot, agents open source ou agents internes deviennent des exécutants raccordés par adaptateurs.

Capacité abstraite	Exigence AgentOps OS	Exemples de mapping
Read codebase	Lire fichiers selon permissions.	Claude Code, Codex, CLI local.
Write diff	Produire patch versionné.	Git worktree, branch, PR.
Run command	Exécuter commande autorisée.	Sandbox executor.
Use instructions	Charger règles projet.	AGENTS.md, CLAUDE.md, .agentops/rules.md.
Subagents	Déléguer rôles spécialisés.	Claude subagents, Codex agents, workers internes.
Hooks	Pré/post-action guardrails.	Claude hooks, CI hooks, Git hooks.
Skills/tools	Capacités déclarées.	MCP, scripts, tool registry.

Le fournisseur est une option d’exécution, pas la constitution du système. AgentOps OS doit pouvoir changer de modèle sans réécrire la gouvernance.

29. API technique cible

Surface d'intégration pour app, IDE, CLI et systèmes tiers

```
POST /v1/projects
GET /v1/projects/{project_id}
POST /v1/projects/{project_id}/missions
GET /v1/missions/{mission_id}
POST /v1/missions/{mission_id}/plan
POST /v1/missions/{mission_id}/approve
POST /v1/missions/{mission_id}/execute
POST /v1/missions/{mission_id}/evaluate
POST /v1/missions/{mission_id}/close

GET /v1/agents
POST /v1/agents/{agent_id}/assign
GET /v1/policies/evaluate?action=...
POST /v1/evidence
GET /v1/audit?mission_id=...
POST /v1/improvements/propose
POST /v1/improvements/{id}/approve
POST /v1/improvements/{id}/apply
```

L'API manipule des missions, actions, preuves, politiques, évaluations et approbations. Elle n'est pas une API de conversation.

30. CLI cible

Contrôle local et scriptable

```
agentops init
agentops scan
agentops mission create --from "Add login page"
agentops mission plan AOS-MIS-2026-0001
agentops mission execute AOS-MIS-2026-0001 --agent coder --autonomy 3
agentops mission evaluate AOS-MIS-2026-0001
agentops approve AOS-MIS-2026-0001 --scope execution
agentops audit tail
agentops memory add --type decision --text "No DB changes without approval"
agentops improve propose --from-last 30d
agentops policy test --action ./sample-action.json
agentops report mission AOS-MIS-2026-0001 --format pdf
```

Le CLI rend AgentOps OS utilisable dans scripts, CI/CD, terminaux et environnements air-gapped. Il doit fonctionner même sans interface web.

31. Intégration IDE

L'IDE comme cockpit d'usine d'intelligence

- Panneau Missions : création, statut, prochaine validation.
- Panneau Agents : rôle actif, permissions, historique de performance.
- Panneau Risques : fichiers sensibles, politiques déclenchées, warnings.
- Panneau Evidence : diff, tests, logs, captures, approbations.
- Panneau Mémoire : règles apprises, décisions, patterns réutilisables.
- Boutons contextuels : expliquer fichier, auditer, générer tests, proposer refactor, créer mission.
- Mode safe review : l'agent commente sans modifier.
- Mode branch sandbox : l'agent agit dans un worktree isolé.

Interface

L'IDE doit rendre visibles les limites de l'agent. L'utilisateur doit toujours savoir ce que l'agent peut lire, écrire, exécuter et proposer.

32. Déploiements et topologies

Local, équipe, entreprise et environnements sensibles

Topologie	Description	Usage
Local solo	Noyau .agentops + CLI + agent local/cloud.	Indépendant, prototype, apprentissage.
Team workspace	Serveur AgentOps + repos partagés + approvals.	Startup, agence, équipe produit.
Enterprise	RBAC, SSO, audit central, politiques managées.	Organisation multi-équipes.
Air-gapped	Modèles locaux, registre interne, pas d’Internet.	Défense, santé, industrie sensible.
Hybrid	Orchestrateur central + exécution locale par projet.	Grandes organisations distribuées.

La même spécification s’applique partout. Ce qui change est la sévérité des politiques, la localisation des modèles, la gestion des secrets et le niveau d’audit.

33. Observabilité

Voir ce que produit vraiment l’usine

Métrique	Définition	Usage
Mission cycle time	Temps entre intention et clôture.	Performance globale.
Human review load	Temps humain passé en validation.	Coût réel.
Agent rework rate	Part des sorties nécessitant correction.	Qualité agentique.
Test failure recovery	Temps de correction après test échoué.	Résilience.
Policy block rate	Actions bloquées ou escaladées.	Sécurité et friction.
Learning yield	Leçons réutilisables par mission.	Capitalisation.
Autonomy incidents	Événements où autonomie a produit risque.	Gouvernance.
Evidence completeness	Missions avec preuves complètes.	Auditabilité.

Un dashboard de productivité qui ignore les corrections humaines donne une mesure trompeuse. L’observabilité doit couvrir les agents, les humains, les outils, les workflows et les policies.

PARTIE IV

Exploitation et gouvernance

34. Workflows opérationnels

Catalogue complet de production

Workflow	But	Points de contrôle
Feature	Créer une nouvelle capacité produit.	Plan, tests, UX review, PR.
Bugfix	Reproduire et corriger un défaut.	Reproduction, test anti-régression.
Refactor	Améliorer structure sans changer comportement.	Diff limité, tests complets.
Security audit	Identifier et corriger risques.	Triage, criticité, validation sécurité.
Migration	Modifier framework, API ou données.	Plan rollback, tests compatibilité.
Documentation	Mettre à jour connaissance.	Source, version, review.
Release	Préparer livraison.	Changelog, CI, approval, monitoring.
Incident response	Diagnostiquer et corriger urgence.	Logs, post-mortem, memory update.
Self-improve	Améliorer le système agentique.	Évaluation avant/après, double approval.

Chaque workflow est un pipeline de sécurité et d'apprentissage. Le workflow self-improve doit être traité comme un changement d'infrastructure, pas comme une simple préférence de prompt.

35. Workflow Feature complet

De l'idée à la fonctionnalité validée

1. Créer mission avec intention, contexte, scope et critères de réussite.
2. L'Architect cartographie fichiers, dépendances, risques et stratégie.
3. Le Planner propose découpage en sous-tâches.
4. Le Reviewer vérifie que le plan est minimal et cohérent.
5. L'humain valide exécution si le niveau d'autonomie l'exige.
6. Le Coder travaille sur branche isolée.
7. Le Tester lance tests obligatoires et ajoute tests si requis.
8. Le Security vérifie secrets, entrées utilisateur, dépendances et surfaces sensibles.
9. Le Documenter met à jour changelog et docs si nécessaire.
10. Le Human Reviewer valide le diff et demande corrections si besoin.
11. La PR est ouverte avec rapport, preuves et risques résiduels.
12. La mémoire est mise à jour avec décisions et patterns utiles.

```
feature.workflow.yml:
required_agents: [planner, architect, coder, tester, reviewer]
optional_agents: [security, documenter]
gates:
- plan_approved
- tests_passed
- risk_below_threshold
- human_review_complete
mandatory_evidence:
- plan.md
- diff.patch
- test-results.json
- mission-report.md
```

36. Workflow Bugfix complet

Corriger sans créer de régression

1. Décrire symptôme, environnement et résultat attendu.
2. Reproduire le bug ou expliquer pourquoi il n'est pas reproductible.
3. Identifier cause probable avec evidence.
4. Modifier le minimum nécessaire.
5. Ajouter ou mettre à jour un test anti-régression.
6. Relancer tests ciblés puis tests globaux selon criticité.
7. Produire rapport de cause racine.
8. Mettre à jour memory/incidents.md si le bug révèle un pattern.

Règle anti-dérive

Un agent ne doit pas corriger à l'aveugle sans reproduction, sauf mission d'urgence explicitement marquée incident.

37. Workflow Self-Improve complet

Capitaliser chaque erreur agentique

Phase	Question	Artefact
Observation	Quelle erreur ou friction se répète ?	log analysis
Hypothèse	Quelle règle ou outil aurait évité cela ?	proposal.md
Simulation	Que se serait-il passé sur missions passées ?	replay report
Évaluation	Le gain est-il mesurable ?	before/after scores
Risque	Le changement peut-il réduire sécurité ou qualité ?	risk assessment
Approbation	Qui peut valider ?	approval log
Application	Comment appliquer et rollback ?	patch + rollback plan
Suivi	Les métriques s'améliorent-elles ?	monitoring window

L'auto-amélioration privilégie les changements explicables : tests ajoutés, workflows clarifiés, règles de permissions, checklists, prompts structurés, sous-agents spécialisés. Les changements opaques ou globaux sont haut risque.

38. Maturité organisationnelle

Passer de l'usage individuel à l'usine gouvernée

Niveau	Nom	Caractéristiques
0	Chat assisté	Copier-coller de prompts, pas d'audit.
1	Agent local	Agent de codage utilisé manuellement.
2	Projet agentisé	.agentops, règles, missions, mémoire locale.
3	Équipe orchestrée	Rôles, approvals, logs, workflows communs.
4	Usine mesurée	Scores, dashboards, evaluation engine, policy engine.
5	Auto-amélioration supervisée	MetaDev, replay, amélioration contrôlée.
6	Entreprise agentique	RBAC, conformité, multi-projets, audit central.

La maturité ne se mesure pas au nombre d'agents mais à la capacité de prouver, gouverner et améliorer leurs sorties.

39. Gouvernance et rôles humains

La nouvelle organisation autour des agents

Rôle humain	Responsabilité
AgentOps Owner	Responsable de la méthode, des standards et de la roadmap.
Technical Owner	Valide architecture, merges sensibles, dette technique.
Security Owner	Valide policies, secrets, réseau, vulnérabilités.
Product Owner	Valide valeur métier et critères d'acceptation.
Release Owner	Valide livraison et rollback.
Auditor	Vérifie preuves, logs, conformité et incidents.
MetaDev Supervisor	Valide auto-améliorations agentiques.

Le rôle humain évolue : moins d'exécution répétitive, plus de cadrage, arbitrage, évaluation, validation et conception de systèmes.

40. Anti-patterns

Ce qu'il faut explicitement éviter

Anti-pattern	Pourquoi c'est dangereux	Correction
Agent tout-puissant	Concentre trop de capacités.	Séparer rôles et permissions.
Pas de preuve	Impossible de vérifier ou apprendre.	Evidence obligatoire.
Mémoire non révisée	Accumule erreurs et préférences obsolètes.	Expiry + revue.
Autonomie par enthousiasme	Décisions basées sur impression.	Scores et incidents.
Prompt comme sécurité	Instruction contournable.	Sandbox + policy engine.
Tests optionnels	Qualité invérifiable.	Gates obligatoires.
Auto-amélioration directe	Risque de dérégulation.	Proposition + simulation + approval.
Vendor lock-in	Dépendance fragile.	Adaptateurs et standards ouverts.

AgentOps OS doit être conçu contre ces anti-patterns dès le départ. Les corriger plus tard coûte plus cher que les prévenir dans le standard.

41. Doctrine qualité 10/10

Fond et forme du système cible

Pilier	Exigence 10/10
Clarté	Tout agent, humain ou outil comprend la mission, les limites et les preuves attendues.
Portabilité	Le noyau .agentops fonctionne dans une app, un IDE, un dépôt Git ou un environnement autonome.
Sécurité	Les limites sont techniques, pas seulement verbales.
Évaluation	Chaque action importante est mesurée selon qualité, risque, coût et confiance.
Mémoire	Le projet apprend sans stocker de secrets ni amplifier des erreurs.
Auto-amélioration	Le système devient meilleur par changement évalué, approuvé et réversible.

La forme sert le fond : documents courts dans le dépôt, rapports lisibles, dashboards orientés décision, logs exploitables, schémas suffisamment stricts pour permettre l’automatisation.

PARTIE V

Annexes réutilisables

A. Annexe A - Mission Report

Rapport généré après exécution agentique

```
# Mission Report - AOS-MIS-2026-0001
```

```
## Résumé
```

```
Objectif atteint : oui
```

```
Risque résiduel : faible
```

```
Autonomie utilisée : niveau 4
```

```
Validation humaine : requise avant merge
```

```
## Changements
```

```
- Ajout validation email dans EmailField.tsx
```

```
- Ajout test EmailField.test.tsx
```

```
- Mise à jour documentation formulaire
```

```
## Preuves
```

```
- diff.patch: sha256:...
```

```
- npm run test: passed
```

```
- npm run typecheck: passed
```

```
- npm run build: passed
```

```
## Risques
```

```
- Aucun changement auth
```

```
- Aucun changement database
```

```
- Aucune nouvelle dépendance
```

```
## Leçon à mémoriser
```

```
Pour toute modification de formulaire, ajouter un test de champ vide et un test de format invalide.
```

B. Annexe B - Schéma JSON Mission

Contrat machine simplifié

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "AgentOps Mission",
  "type": "object",
  "required": ["mission_id", "title", "intent", "scope", "success_criteria",
    "autonomy_level"],
  "properties": {
    "mission_id": {"type": "string"},
    "title": {"type": "string"},
    "intent": {"type": "string"},
    "context": {"type": "object"},
    "scope": {
      "type": "object",
      "properties": {
        "include": {"type": "array", "items": {"type": "string"}},
        "exclude": {"type": "array", "items": {"type": "string"}}
      }
    },
    "constraints": {"type": "object"},
    "success_criteria": {"type": "array", "items": {"type": "string"}},
    "risk_level": {"enum": ["low", "medium", "high", "critical"]},
    "autonomy_level": {"type": "integer", "minimum": 0, "maximum": 9}
  }
}
```

C. Annexe C - Checklists

Contrôles minimaux

Checklist	Questions
Avant exécution	Mission claire ? scope limité ? risque classé ? approbation requise ?
Avant écriture	Fichiers autorisés ? branche isolée ? rollback possible ?
Avant test	Commandes autorisées ? réseau nécessaire ? secrets masqués ?
Avant merge	Diff lisible ? tests passés ? revue humaine ? documentation ?
Avant auto-amélioration	Gain prouvé ? simulation ? double approval ? rollback ?
Après incident	Cause racine ? policy manquante ? mémoire mise à jour ?

D. Annexe D - Références

Sources utiles pour ancrer la spécification

Ces références ne définissent pas AgentOps OS. Elles ancrent le contexte technique : agents de codage capables de lire/éditer/exécuter, sandboxing, permissions, mémoire, hooks, instructions projet et limites observées de productivité.

[R1] Anthropic - Claude Code overview

Claude Code est décrit comme un outil agentique qui lit le codebase, édite des fichiers, lance des commandes et s'intègre aux outils de développement.

<https://code.claude.com/docs/en/overview>

[R2] Anthropic - Claude Code settings / sandboxing

La documentation décrit les réglages de permissions et de sandboxing.

<https://docs.anthropic.com/en/docs/claude-code/settings>

[R3] Anthropic - Hooks reference

Les hooks permettent d'exécuter automatiquement commandes, endpoints ou prompts à différents moments du cycle de vie.

<https://docs.anthropic.com/en/docs/claude-code/hooks>

[R4] Anthropic - Claude Code memory

La mémoire de projet et l'auto-memory permettent d'intégrer corrections et habitudes du projet.

<https://docs.anthropic.com/en/docs/claude-code/memory>

[R5] OpenAI - Codex web

Codex est présenté comme un agent de codage capable de lire, éditer et exécuter du code dans des environnements cloud.

<https://developers.openai.com/codex/cloud>

[R6] OpenAI - Agent internet access

Par défaut, l'accès Internet de l'agent est bloqué pendant la phase agent ; l'activation augmente certains risques.

<https://developers.openai.com/codex/cloud/internet-access>

[R7] OpenAI - AGENTS.md

Codex lit les fichiers AGENTS.md avant de travailler afin d'appliquer des instructions globales et spécifiques au projet.

<https://developers.openai.com/codex/guides/agents-md>

[R8] OpenAI - Agent approvals & security

Codex utilise sandbox et politiques d'approbation pour encadrer l'action locale.

<https://developers.openai.com/codex/agent-approvals-security>

[R9] Robeyns, Szummer, Aitchison - A Self-Improving Coding Agent, arXiv 2025

Le papier montre qu'un système agentique doté d'outils de codage peut s'éditer et améliorer ses performances sur des benchmarks.

<https://arxiv.org/abs/2504.15228>

[R10] METR - Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity, 2025

Étude RCT indiquant que des développeurs open-source expérimentés ont pris 19% plus de temps avec des outils IA dans le contexte étudié.

<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>